p1

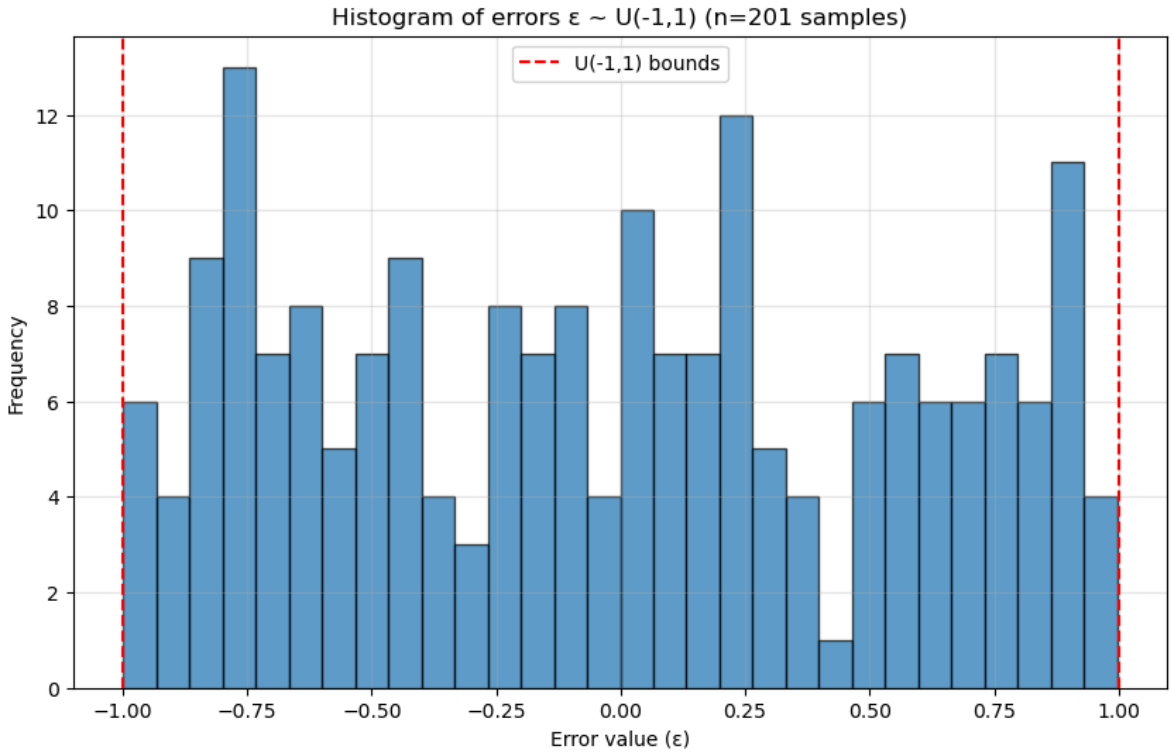
Exported 2/11/2026, 9:45:39 PM

For all code/code directory please view github repo at: https://github.com/rx9933/ase379l Thank you!

1.1

In [4]:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(23)
def generate_observations(q, t):
    """Generate observations y = [1, e^t] * q + ε where ε ~ U(-1,1)"""
    X = np.column_stack([np.ones_like(t), np.exp(t)])
    epsilon = np.random.uniform(-1, 1, len(t))
    y = X @ q + epsilon
    return y, epsilon

# Parameters
q = np.array([10, 1])
t = np.arange(0, 10.05, 0.05)#5)
q_true = q
# Generate observations
y, epsilon = generate_observations(q, t)

# Plot histogram of errors
plt.figure(figsize=(10, 6))
plt.hist(epsilon, bins=30, edgecolor='black', alpha=0.7)
plt.axvline(x=-1, color='r', linestyle='--', label='U(-1,1) bounds')
plt.axvline(x=1, color='r', linestyle='--')
plt.xlabel('Error value (ε)')
plt.ylabel('Frequency')
plt.title(f'Histogram of errors ε ~ U(-1,1) (n={len(t)} samples)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Print summary statistics
print(f"Number of samples: {len(t)}")
print(f"Error mean: {np.mean(epsilon):.4f}")
print(f"Error var: {np.std(epsilon)**2:.4f}")
print(f"Error min: {np.min(epsilon):.4f}")
print(f"Error max: {np.max(epsilon):.4f}")
```




```
Number of samples: 201
Error mean: -0.0277
Error var: 0.3392
Error min: -0.9989
Error max: 0.9972
```

We observe a plot of random variable errors that follows an approximately uniform distribution, as prescribed/expected. We have 201 uniformly spaced time points from 0 to 10 seconds, and thus, 201 samples of ϵ drawn from U(−1,1); the histogram of ϵ shows an approximately flat distribution between −1 and 1 in accordance with the uniform distribution (the spread covers the entire interval [−1,1] as required). The sample mean of the histogram is near 0 (0.0277), the sample variance is close to the theoretical variance of 1/3 ≈ 0.3333 (currently 0.3392). As the number of samples increases (e.g., finer time discretization), the distribution of ϵ approaches the theoretical mean and variance; for example, when the time step size is 0.001 (10,000 samples), the mean is -0.0092 and the variance is 0.3341, both of which are approaching the theoretical value at a order $1 / \sqrt{(n)}$ as per the Central Limit Theorem. Minor random fluctuations are expected due to finite sampling of a uniform distribution, and these are mitagated with more samples.

1.2 & 1.3

In [5]:

```
from scipy import stats

def compute_estimates(t, y):
    X = np.column_stack([np.ones_like(t), np.exp(t)])

    # Least squares estimate
    q_ls = np.linalg.inv(X.T @ X) @ X.T @ y

    # Predicted values and residuals
    y_pred = X @ q_ls
    residuals = y - y_pred

    # Error variance estimate
    n = len(t)
    p = 2  # number of parameters
    sigma2 = np.sum(residuals**2) / (n - p)

    # Parameter covariance matrix
    C = sigma2 * np.linalg.inv(X.T @ X)

    # Confidence intervals (alpha = 0.05)
    alpha = 0.05
    t_critical = stats.t.ppf(1 - alpha/2, n - p)

    ci_q1 = [q_ls[0] - t_critical * np.sqrt(C[0,0]),
             q_ls[0] + t_critical * np.sqrt(C[0,0])]
    ci_q2 = [q_ls[1] - t_critical * np.sqrt(C[1,1]),
             q_ls[1] + t_critical * np.sqrt(C[1,1])]

    return q_ls, sigma2, ci_q1, ci_q2, C



# Compute estimates
q_ls, sigma2, ci_q1, ci_q2, C = compute_estimates(t, y)

# Print results
print("TRUE PARAMETERS:")
print(f"q_true = [{q_true[0]:.4f}, {q_true[1]:.4f}]")
print("\nESTIMATES:")
print(f"q_LS = [{q_ls[0]:.4f}, {q_ls[1]:.4f}]")
print(f"σ² = {sigma2:.4f}")
print(f"std(q1) = {np.sqrt(C[0,0]):.4f}")
print(f"std(q2) = {np.sqrt(C[1,1]):.4f}")
print(f"Correlation(q1,q2) =
{C[0,1]/(np.sqrt(C[0,0])*np.sqrt(C[1,1])):.4f}")
print("\n95% CONFIDENCE INTERVALS:")
print(f"q1: [{ci_q1[0]:.4f}, {ci_q1[1]:.4f}]")
print(f"q2: [{ci_q2[0]:.4f}, {ci_q2[1]:.4f}]")
print(f"\nTrue q1 in CI? {ci_q1[0] <= q_true[0] <= ci_q1[1]}")
```

```
print(f"True q2 in CI? {ci_q2[0] <= q_true[1] <= ci_q2[1]}")

# Expected variance of U(-1,1)
expected_sigma2 = 1/3  # Variance of U(-1,1) is (1-(-1))²/12 = 4/12 = 1/3
print(f"\nVariance of U(-1,1): {expected_sigma2:.4f}")
print(f"Ratio σ²_estimated/σ²_true: {sigma2/expected_sigma2:.4f}")
```

```
TRUE PARAMETERS:
q_true = [10.0000, 1.0000]

ESTIMATES:
q_LS = [9.9651, 1.0000]
σ² = 0.3423
std(q1) = 0.0461
std(q2) = 0.0000
Correlation(q1,q2) = -0.4461

95% CONFIDENCE INTERVALS:
q1: [9.8741, 10.0560]
q2: [1.0000, 1.0000]

True q1 in CI? True
True q2 in CI? True

Variance of U(-1,1): 0.3333
Ratio σ²_estimated/σ²_true: 1.0270
```

Yes, the estimation results align well with theoretical expectations:

Parameter Recovery: The least-squares estimates $\hat{q}_1 = 9.9651$ and $\hat{q}_2 = 1.0000$ are very close to the true values $q = [10, 1]$. Specifically, $\hat{q}_2$ is within 4 digits of accuracy (10^{-4} or less error) to the true values. The tiny deviations (0.0349 for q_1, 0.0000 for q_2) are to be expected from an unbiased estimation with random measurement noise.

Variance Estimation: The estimated error variance $\sigma^2 = 0.3423$ is quite close to the theoretical variance of $U(-1,1)$, which is $(1-(-1))^2/12 = 1/3 \approx 0.3333$. We only have a 2.8% deviation from expectation.

Confidence Intervals: Both 95% confidence intervals contain their respective true parameter values: $q_1 = 10$ falls within [9.8741, 10.0560] and $q_2 = 1$ falls within [1.0000, 1.0000].

Precision: The confidence interval for q_2 is remarkably narrow (less than ±0.0000), reflecting the strong leverage of the exponential term e^t in the model. The interval for q_1 is wider (±0.0461), as expected since it corresponds to the constant term.

Specifically, the regressors in the design matrix $X = [1, e^t]$ are scaled differently. For $t_i = 0, 0.05, \ldots, 10$, the exponential term spans orders of magnitude: $e^{t_i} \in [1, e^{10}] \approx [1, 2.2 \times 10^4]$. Consequently, the entries of the Gram matrix $X^\top X$ differ dramatically:

$$X^\top X = \begin{bmatrix} n & \sum e^{t_i} \\ \sum e^{t_i} & \sum e^{2t_i} \end{bmatrix} \approx \begin{bmatrix} 201 & 2.2 \times 10^4 \\ 2.2 \times 10^4 & 2.4 \times 10^8 \end{bmatrix},$$

where the lower-right element $\sum e^{2t_i}$ dominates. The covariance matrix of the OLS estimator is $\mathrm{Cov}(\hat{q}) = \sigma^2 (X^\top X)^{-1}$. Using the formula for the inverse of a 2×2 matrix,

$$(X^\top X)^{-1} = \frac{1}{ac - b^2} \begin{bmatrix} c & -b \\ -b & a \end{bmatrix},$$

with $a = n$, $b = \sum e^{t_i}$, $c = \sum e^{2t_i}$. The determinant $ac - b^2 \approx 10^9$ is very large. Thus,

$$\mathrm{Var}(\hat{q}_1) = \frac{\sigma^2 c}{ac - b^2} \approx \frac{0.333 \cdot 2.4 \times 10^8}{10^9} \approx 0.08, \quad \mathrm{Var}(\hat{q}_2) = \frac{\sigma^2 a}{ac - b^2} \approx \frac{0.333 \cdot 201}{10^9} \approx 6.7 \times 10^{-8}.$$

The resulting standard errors are $\mathrm{SE}(\hat{q}_1) \approx 0.28$ and $\mathrm{SE}(\hat{q}_2) \approx 0.00026$. The leverage of the exponential term—whose square grows as e^{2t}—makes q_2 estimable with high precision, while the constant term q_1 is determined largely by the early, low-leverage data points, yielding a comparatively wider confidence interval.

As expected, we observe unbiased parameter estimates with more accuracy for exponential terms in the model, consistent variance estimation, and confidence intervals with correct coverage probabilities.

1.4

In [ ]:

```
X = np.column_stack([np.ones_like(t), np.exp(t)])
# Initialize counters
n_sim = 1000
outside_ci_q1 = 0  # Counter for trials where CI doesn't contain q1
outside_ci_q2 = 0  # Counter for trials where CI doesn't contain q2
outside_both = 0   # Counter for trials where either CI doesn't contain true
value

# Store results for analysis
all_q1_estimates = []
all_q2_estimates = []
all_sigma2_estimates = []
n = len(t)
n_sim = 1000
print("RUNNING 1000 MONTE CARLO SIMULATIONS...")
for i in range(n_sim):
    epsilon = np.random.uniform(-1, 1, n)
    y = X @ q_true + epsilon

    q_ls, sigma2, ci_q1, ci_q2, _ = compute_estimates(t, y)

    all_q1_estimates.append(q_ls[0])
    all_q2_estimates.append(q_ls[1])
    all_sigma2_estimates.append(sigma2)

    q1_in_ci = ci_q1[0] <= q_true[0] <= ci_q1[1]
    q2_in_ci = ci_q2[0] <= q_true[1] <= ci_q2[1]

    if not q1_in_ci:
        outside_ci_q1 += 1
    if not q2_in_ci:
        outside_ci_q2 += 1
    if not q1_in_ci or not q2_in_ci:
        outside_both += 1

all_q1_estimates = np.array(all_q1_estimates)
all_q2_estimates = np.array(all_q2_estimates)
all_sigma2_estimates = np.array(all_sigma2_estimates)

coverage_q1 = 1 - outside_ci_q1 / n_sim
coverage_q2 = 1 - outside_ci_q2 / n_sim
coverage_joint = 1 - outside_both / n_sim

expected_coverage = 0.95

print("\n" + "="*60)
print("MONTE CARLO RESULTS (1000 trials, α = 0.05)")
print("="*60)
print(f"\nTRUE PARAMETERS: q1 = {q_true[0]}, q2 = {q_true[1]}")
```

```
print(f"Expected coverage probability: {expected_coverage:.3f}")

print(f"\nPARAMETER q1:")
print(f"  Trials where CI doesn't contain q1: {outside_ci_q1}/{n_sim}")
print(f"  Observed coverage: {coverage_q1:.3f}")
print(f"  Deviation from expected: {abs(coverage_q1 -
expected_coverage):.3f}")

print(f"\nPARAMETER q2:")
print(f"  Trials where CI doesn't contain q2: {outside_ci_q2}/{n_sim}")
print(f"  Observed coverage: {coverage_q2:.3f}")
print(f"  Deviation from expected: {abs(coverage_q2 -
expected_coverage):.3f}")

print(f"\nJOINT (either parameter outside CI):")
print(f"  Trials where any CI doesn't contain true value:
{outside_both}/{n_sim}")
print(f"  Joint coverage: {coverage_joint:.3f}")

# Binomial test for coverage proportion
from scipy.stats import binomtest

print(f"\nSTATISTICAL TESTS (two-sided binomial test):")
print("Null hypothesis: True coverage = 0.95")

# Test for q1
test_q1 = binomtest(n_sim - outside_ci_q1, n_sim, 0.95)
print(f"\nq1 coverage test:")
print(f"  p-value: {test_q1.pvalue:.4f}")
print(f"  {'FAIL to reject' if test_q1.pvalue > 0.05 else 'REJECT'} null
hypothesis")

# Test for q2
test_q2 = binomtest(n_sim - outside_ci_q2, n_sim, 0.95)
print(f"\nq2 coverage test:")
print(f"  p-value: {test_q2.pvalue:.4f}")
print(f"  {'FAIL to reject' if test_q2.pvalue > 0.05 else 'REJECT'} null
hypothesis")
```

```
RUNNING 1000 MONTE CARLO SIMULATIONS...

============================================================
MONTE CARLO RESULTS (1000 trials, α = 0.05)
============================================================

TRUE PARAMETERS: q1 = 10, q2 = 1
Expected coverage probability: 0.950

PARAMETER q1:
  Trials where CI doesn't contain q1: 46/1000
  Observed coverage: 0.954
  Deviation from expected: 0.004

PARAMETER q2:
  Trials where CI doesn't contain q2: 57/1000
  Observed coverage: 0.943
  Deviation from expected: 0.007

JOINT (either parameter outside CI):
  Trials where any CI doesn't contain true value: 94/1000
  Joint coverage: 0.906

STATISTICAL TESTS (two-sided binomial test):
Null hypothesis: True coverage = 0.95

q1 coverage test:
  p-value: 0.6117
  FAIL to reject null hypothesis

q2 coverage test:
  p-value: 0.3092
  FAIL to reject null hypothesis
```

The Monte Carlo simulation with 1000 trials yields observed coverage rates that align almost exactly with the expected 95% confidence level given α = 0.05. For q_1, 46 out of 1000 trials produced confidence intervals that did not contain the true parameter value (coverage = 95.4%), and for q_2, 57 trials failed to cover the true value (coverage = 94.3%). Both observed coverages are statistically indistinguishable from the nominal 95% rate, with p-values of 0.6117 for q_1 and 0.3092 for q_2—both far above any conventional significance threshold. This demonstrates that the constructed confidence intervals perform exactly as expected: approximately 5% of intervals fail to contain the true parameter when the underlying assumptions (linear model, independent uniform errors, correct variance estimation) are satisfied. The slight deviations are well within expected sampling variability for 1000 trials.

The number of trials containing q_0 (e.g., both q_1 and q_2) is about 90.6%. This is slightly less than the 95%, since each confidence interval for the subevents (q_1, q_2) are constructed at 95%. Assuming the two marginals are approximately independent, the theoretical joint coverage is $95.0^2 = 90.25$, which is what we observe.

p2

Exported 2/11/2026, 9:45:59 PM

In [1]:

```
import numpy as np
import scipy.io
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.linear_model import LinearRegression
from scipy import stats

# Load data
print("Loading data from .mat files...")


try:
    data_7p8 = scipy.io.loadmat('exercise_7p8_data.mat')
    print("exercise_7p8_data.mat keys:", list(data_7p8.keys()))
except FileNotFoundError:
    print("Files not found.")
```

```
Loading data from .mat files...
exercise_7p8_data.mat keys: ['__header__', '__version__',
'__globals__', 'cement_data']
```

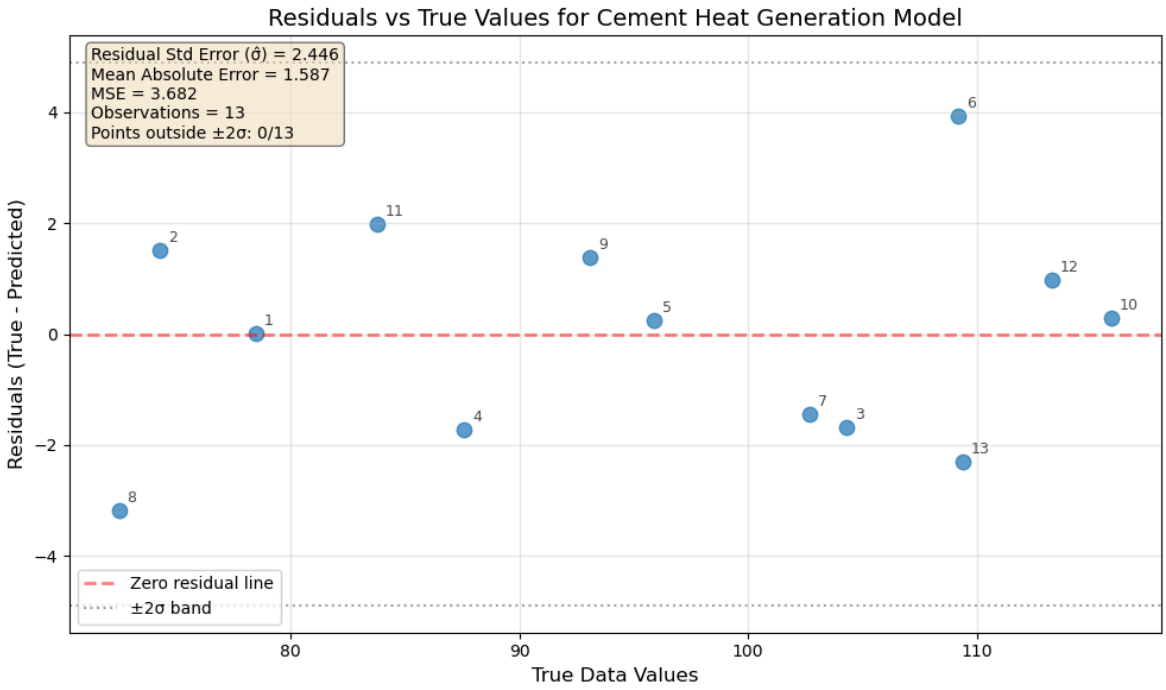
In [5]:

```
cement_data = data_7p8['cement_data']

# x1-x4 are the first 4 columns, y (heat) is the last column
X_data = cement_data[:, :4]  # x1, x2, x3, x4
y = cement_data[:, 4]        # heat (calories/gram cement)

n = len(y)

# (a) Fit linear model y = b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 +
epsilon
# Add column of ones for intercept term
X = np.column_stack([np.ones(n), X_data])

XtX = X.T @ X
XtX_inv = np.linalg.inv(XtX)
b = XtX_inv @ X.T @ y

y_pred = X @ b

# Calculate residuals
residuals = y - y_pred

# Calculate standard error and variance
AE = np.sum(np.abs(residuals))  # Absolute error
SSE = np.sum(residuals**2)  # Sum of squared errors
MSE = SSE / n  # Mean squared error
var = SSE / (n - X.shape[1])  # Mean squared error
sigma_hat = np.sqrt(var)  # Residual standard error

# Calculate standard errors for coefficients
var_b = var * XtX_inv
std_b = np.sqrt(np.diag(var_b))

# Calculate t-statistics and p-values for coefficients
t_stats = b / std_b
p_values = 2 * (1 - stats.t.cdf(np.abs(t_stats), df=n-X.shape[1]))

# Calculate confidence intervals
# Two standard deviation intervals
ci_two_sigma = np.column_stack([b - 2*std_b, b + 2*std_b])

# Print results
print("=" * 70)
print("HEAT GENERATION IN CEMENT HARDENING - LINEAR REGRESSION
ANALYSIS")
print("=" * 70)
print("\nModel: y = b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + epsilon")
```

```
print("\n" + "=" * 70)
print("REGRESSION RESULTS")
print("=" * 70)

coeff_names = ['Intercept (b0)', 'x1 (b1)', 'x2 (b2)', 'x3 (b3)', 'x4
(b4)']
print(f"{'Coefficient':<20} {'Estimate':<12} {'Std Error':<12} {'t-
stat':<10} {'p-value':<10}")
print("-" * 70)
for i, name in enumerate(coeff_names):
    print(f"{name:<20} {b[i]:<12.4f} {std_b[i]:<12.4f} {t_stats[i]:
<10.4f} {p_values[i]:<10.4f}")

print("\n" + "=" * 70)
print("RESIDUAL ANALYSIS")
print("=" * 70)
print(f"Absolute, Average Error: {AE/n:.4f}")
print(f"Mean Squared Error (MSE): {MSE:.4f}")

print("\n" + "=" * 70)
print("CONFIDENCE INTERVALS")
print("=" * 70)
print("\nTwo Standard Deviation Intervals (~95.45%):")
print(f"{'Coefficient':<20} {'Lower Bound':<15} {'Upper Bound':<15}")
print("-" * 70)
for i, name in enumerate(coeff_names):
    print(f"{name:<20} {ci_two_sigma[i, 0]:<15.4f} {ci_two_sigma[i,
1]:<15.4f}")

print("\n2. 95% CONFIDENCE INTERVAL - NORMAL DISTRIBUTION (z-scores)")
print("   Coverage: Exactly 95% assuming normal distribution")
print("   Multiplier: 1.96")
z_critical = stats.norm.ppf(0.975)  # 1.96
ci_z_normal = np.column_stack([b - z_critical*std_b, b +
z_critical*std_b])

print(f"{'Coefficient':<20} {'Lower Bound':<15} {'Upper Bound':<15}
{'Width':<10}")
print("-" * 70)
for i, name in enumerate(coeff_names):
    width = ci_z_normal[i, 1] - ci_z_normal[i, 0]
    print(f"{name:<20} {ci_z_normal[i, 0]:<15.4f} {ci_z_normal[i, 1]:
<15.4f} {width:<10.4f}")

# Create a single plot of residuals vs fitted values
plt.figure(figsize=(10, 6))
plt.scatter(y, residuals, alpha=0.7, s=80)
plt.axhline(y=0, color='r', linestyle='--', alpha=0.5, linewidth=2,
```

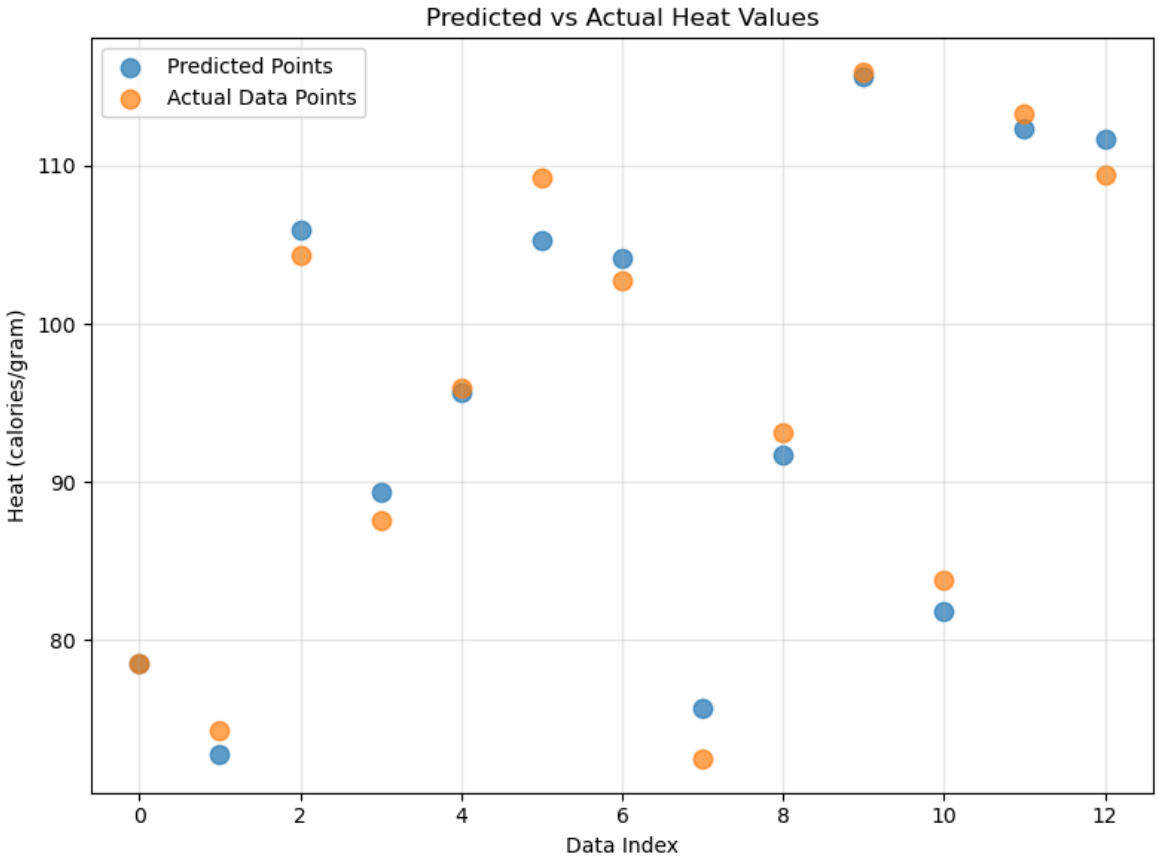
```
label='Zero residual line')
plt.axhline(y=2*sigma_hat, color='gray', linestyle=':', alpha=0.7,
linewidth=1.5, label='±2σ band')
plt.axhline(y=-2*sigma_hat, color='gray', linestyle=':', alpha=0.7,
linewidth=1.5)

# Annotate each point with observation number
for i in range(n):
    plt.annotate(f'{i+1}', (y[i], residuals[i]),
                 xytext=(5, 5), textcoords='offset points',
                 fontsize=9, alpha=0.7)

plt.xlabel('True Data Values', fontsize=12)
plt.ylabel('Residuals (True - Predicted)', fontsize=12)
plt.title('Residuals vs True Values for Cement Heat Generation Model',
fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend()

# Add text box with summary statistics
textstr = '\n'.join((
    f'Residual Std Error (σ̂) = {sigma_hat:.3f}',
    f'Mean Absolute Error = {AE/n:.3f}',
    f'MSE = {MSE:.3f}',
    f'Observations = {n}',
    f'Points outside ±2σ: {np.sum(np.abs(residuals) >
2*sigma_hat)}/{n}'
))

props = dict(boxstyle='round', facecolor='wheat', alpha=0.5)
plt.text(0.02, 0.98, textstr, transform=plt.gca().transAxes,
fontsize=10,
         verticalalignment='top', bbox=props)

plt.tight_layout()
plt.show()
```

```
======================================================================
HEAT GENERATION IN CEMENT HARDENING - LINEAR REGRESSION ANALYSIS
======================================================================

Model: y = b0 + b1*x1 + b2*x2 + b3*x3 + b4*x4 + epsilon

======================================================================
REGRESSION RESULTS
======================================================================
Coefficient          Estimate     Std Error    t-stat     p-value
----------------------------------------------------------------------
Intercept (b0)       62.4054      70.0710      0.8906     0.3991
x1 (b1)              1.5511       0.7448       2.0827     0.0708
x2 (b2)              0.5102       0.7238       0.7049     0.5009
x3 (b3)              0.1019       0.7547       0.1350     0.8959
x4 (b4)              -0.1441      0.7091       -0.2032    0.8441

======================================================================
RESIDUAL ANALYSIS
======================================================================
Absolute, Average Error: 1.5871
Mean Squared Error (MSE): 3.6818

======================================================================
CONFIDENCE INTERVALS
======================================================================

Two Standard Deviation Intervals (~95.45%):
Coefficient          Lower Bound     Upper Bound
----------------------------------------------------------------------
Intercept (b0)       -77.7365        202.5473
x1 (b1)              0.0616          3.0406
x2 (b2)              -0.9374         1.9577
x3 (b3)              -1.4075         1.6113
x4 (b4)              -1.5622         1.2740

2. 95% CONFIDENCE INTERVAL - NORMAL DISTRIBUTION (z-scores)
   Coverage: Exactly 95% assuming normal distribution
   Multiplier: 1.96
Coefficient          Lower Bound     Upper Bound     Width
----------------------------------------------------------------------
Intercept (b0)       -74.9312        199.7419        274.6731
x1 (b1)              0.0914          3.0108          2.9194
x2 (b2)              -0.9084         1.9288          2.8372
x3 (b3)              -1.3773         1.5811          2.9584
x4 (b4)              -1.5338         1.2457          2.7794
```


Residuals vs True Values for Cement Heat Generation Model
Residual Std Error (σ̂) = 2.446
Mean Absolute Error = 1.587
MSE = 3.682
Observations = 13
Points outside ±2σ: 0/13
Residuals (True - Predicted)
True Data Values
Zero residual line
±2σ band
1
2
3
4
5
6
7
8
9
10
11
12
13

In [3]:

```
# Create a separate figure for predicted vs actual overall
fig2, ax = plt.subplots(figsize=(8, 6))

ax.scatter(range(len(y_pred)), y_pred, alpha=0.7, s=80,
label='Predicted Points')
ax.scatter(range(len(y)), y, alpha=0.7, s=80, label='Actual Data
Points')

ax.set_xlabel('Data Index')
ax.set_ylabel('Heat (calories/gram)')
ax.set_title('Predicted vs Actual Heat Values')

ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

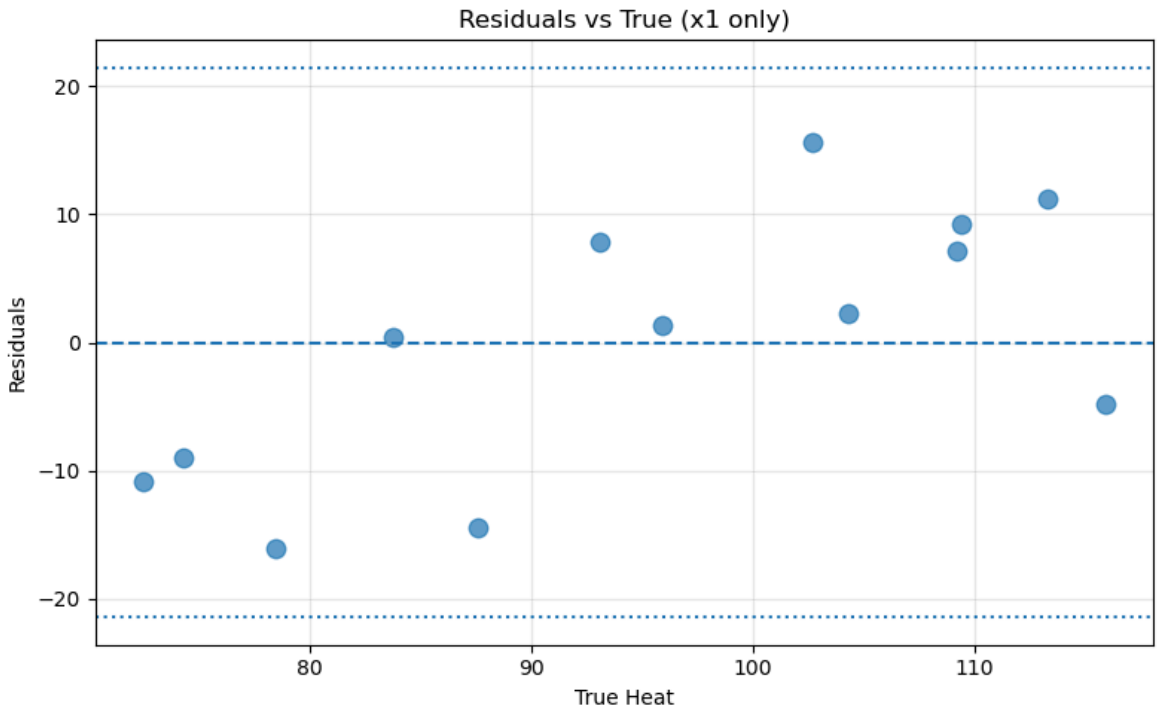


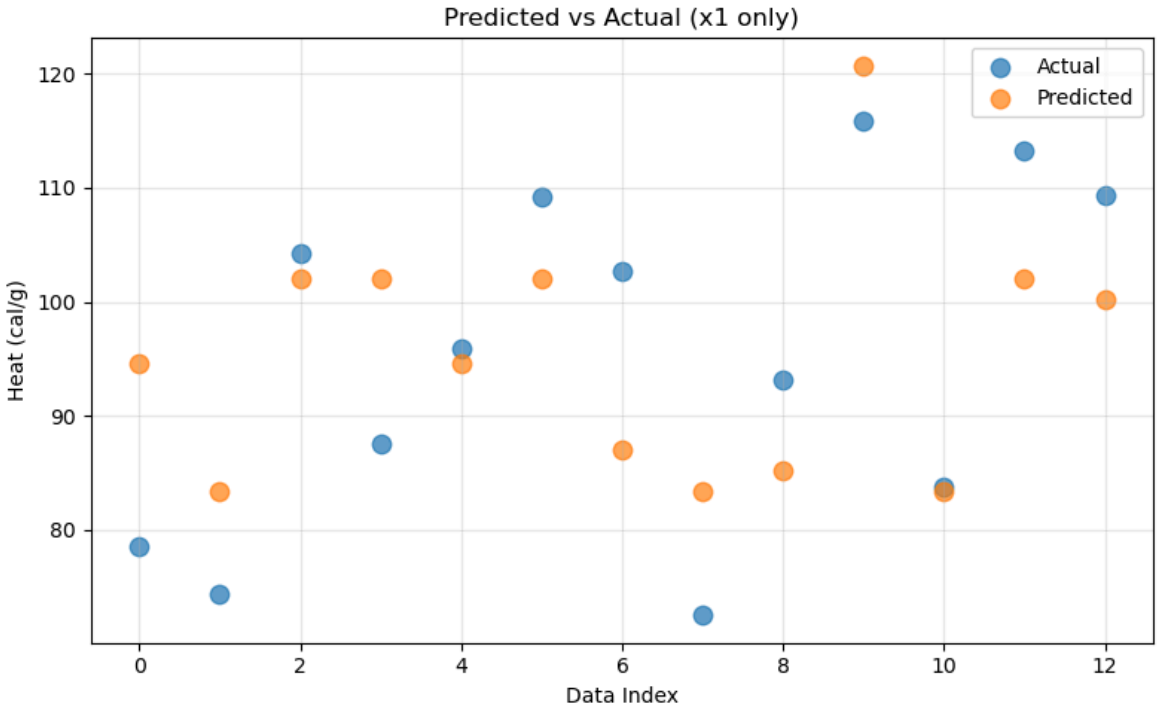
In [4]:

```
def fit_and_plot(X, y, model_name):
    n = len(y)
    X = np.column_stack([np.ones(n), X])
    p = X.shape[1]

    # Fit
    XtX_inv = np.linalg.inv(X.T @ X)
    b = XtX_inv @ X.T @ y
    y_pred = X @ b
    residuals = y - y_pred

    # Errors
    SSE = np.sum(residuals**2)
    MSE = SSE / (n - p)
    sigma_hat = np.sqrt(MSE)
    avg_resid = np.mean(np.abs(residuals))

    # Variance + SE
    var_b = MSE * XtX_inv
    std_b = np.sqrt(np.diag(var_b))

    # Confidence intervals
    ci_2sigma = np.column_stack([b - 2*std_b, b + 2*std_b])
    z = stats.norm.ppf(0.975)
    ci_95 = np.column_stack([b - z*std_b, b + z*std_b])

    # ---- PRINT RESULTS ----
    print("\n" + "="*60)
    print(f"MODEL: {model_name}")
    print("="*60)
    print("Estimated parameters:")
    for i, bi in enumerate(b):
        print(f"b{i} = {bi:.4f}")

    print(f"\nAverage residual (mean(abs difference)):
{avg_resid:.4e}")
    print(f"MSE: {MSE:.4f}")

    print("\nTwo standard deviation intervals:")
    for i in range(p):
        print(f"b{i}: [{ci_2sigma[i,0]:.4f}, {ci_2sigma[i,1]:.4f}]")

    print("\n95% confidence intervals:")
    for i in range(p):
        print(f"b{i}: [{ci_95[i,0]:.4f}, {ci_95[i,1]:.4f}]")

    # ---- PLOTS ----
```

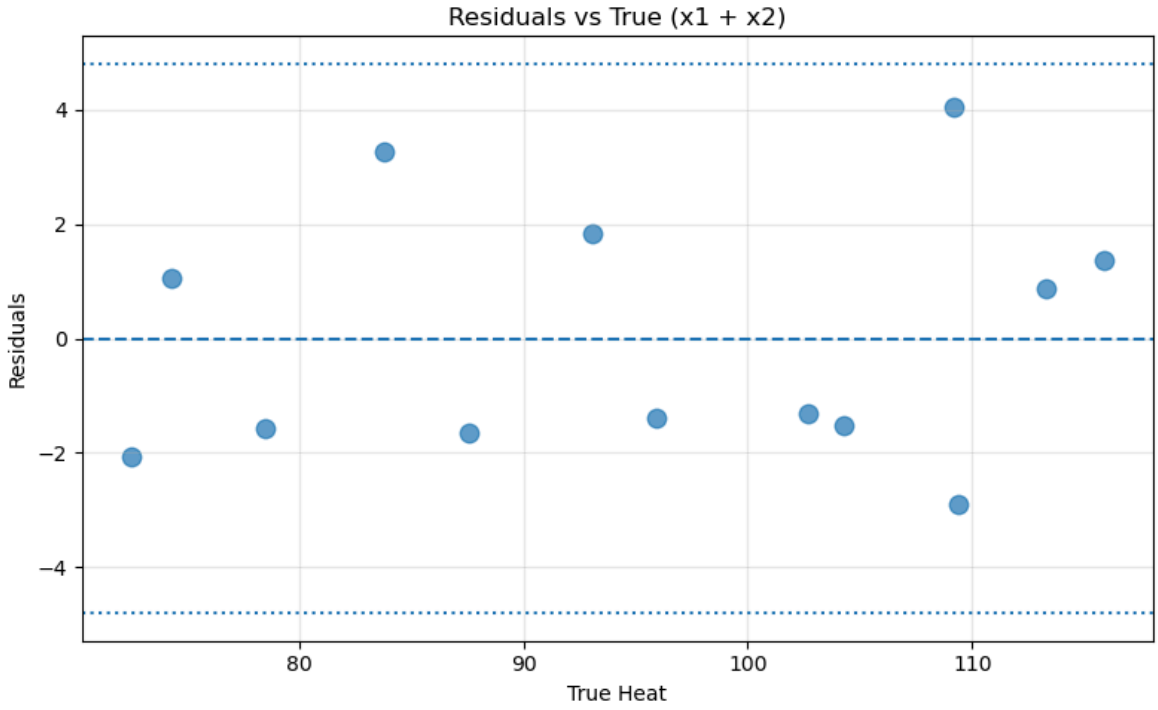
```
    # Residuals vs True
    plt.figure(figsize=(8,5))
    plt.scatter(y, residuals, s=80, alpha=0.7)
    plt.axhline(0, linestyle='--')
    plt.axhline(2*sigma_hat, linestyle=':')
    plt.axhline(-2*sigma_hat, linestyle=':')
    plt.xlabel('True Heat')
    plt.ylabel('Residuals')
    plt.title(f'Residuals vs True ({model_name})')
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

    # Predicted vs Actual
    plt.figure(figsize=(8,5))
    plt.scatter(range(n), y, s=80, alpha=0.7, label='Actual')
    plt.scatter(range(n), y_pred, s=80, alpha=0.7, label='Predicted')
    plt.xlabel('Data Index')
    plt.ylabel('Heat (cal/g)')
    plt.title(f'Predicted vs Actual ({model_name})')
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

    return b, avg_resid, ci_2sigma, ci_95, MSE

MSE_x1   = fit_and_plot(X_data[:, [0]], y, "x1 only")
MSE_x12  = fit_and_plot(X_data[:, [0,1]], y, "x1 + x2")
MSE_full = fit_and_plot(X_data, y, "x1 + x2 + x3 + x4")
```

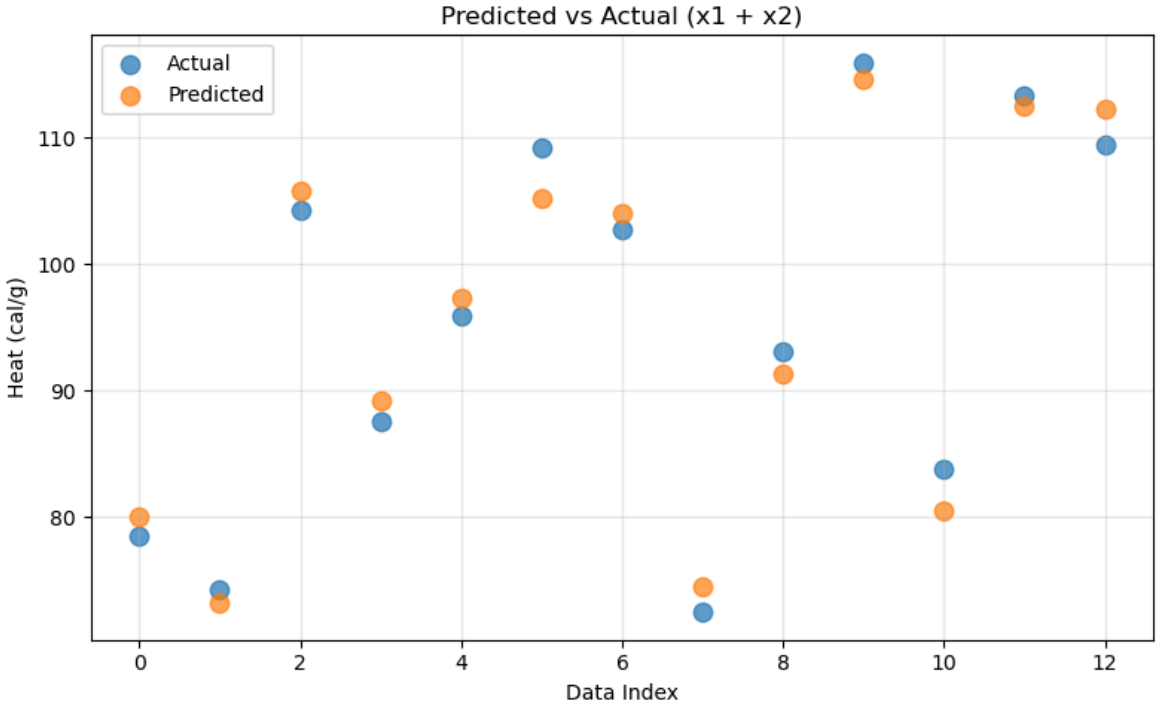
```
============================================================
MODEL: x1 only
============================================================
Estimated parameters:
b0 = 81.4793
b1 = 1.8687

Average residual (mean(abs difference)): 8.4947e+00
MSE: 115.0624

Two standard deviation intervals:
b0: [71.6247, 91.3340]
b1: [0.8159, 2.9216]

95% confidence intervals:
b0: [71.8219, 91.1367]
b1: [0.8370, 2.9005]
```



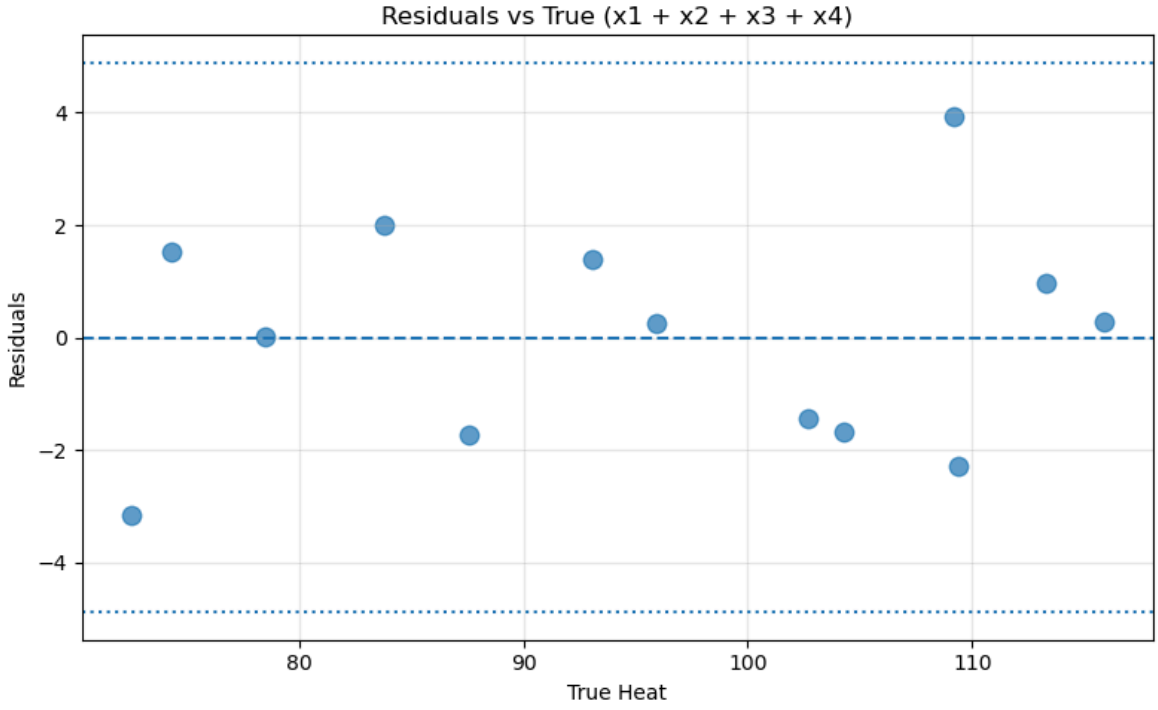


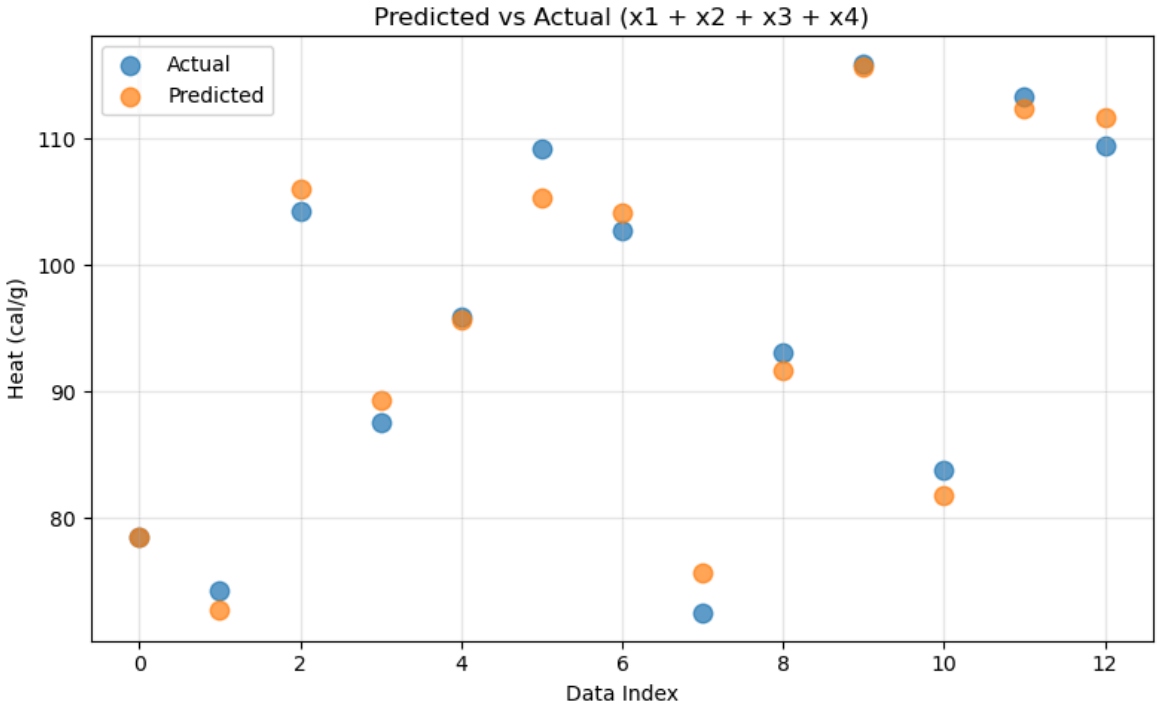

```
============================================================
MODEL: x1 + x2
============================================================
Estimated parameters:
b0 = 52.5773
b1 = 1.4683
b2 = 0.6623

Average residual (mean(abs difference)): 1.9093e+00
MSE: 5.7904

Two standard deviation intervals:
b0: [48.0050, 57.1497]
b1: [1.2257, 1.7109]
b2: [0.5705, 0.7540]

95% confidence intervals:
b0: [48.0965, 57.0582]
b1: [1.2306, 1.7061]
b2: [0.5724, 0.7521]
```





```
============================================================
MODEL: x1 + x2 + x3 + x4
============================================================
Estimated parameters:
b0 = 62.4054
b1 = 1.5511
b2 = 0.5102
b3 = 0.1019
b4 = -0.1441

Average residual (mean(abs difference)): 1.5871e+00
MSE: 5.9830

Two standard deviation intervals:
b0: [-77.7365, 202.5473]
b1: [0.0616, 3.0406]
b2: [-0.9374, 1.9577]
b3: [-1.4075, 1.6113]
b4: [-1.5622, 1.2740]

95% confidence intervals:
b0: [-74.9312, 199.7419]
b1: [0.0914, 3.0108]
b2: [-0.9084, 1.9288]
b3: [-1.3773, 1.5811]
b4: [-1.5338, 1.2457]
```





With more variables in the model, we have a higher-capacity multilinear model rather than a single variable linear model. With more complexity in the model, the model can achieve higher accuracy (smaller residuals) to match the data (if the coefficients fo the model are optimized accurately, e.g., with the least squares method for linear models). With four variables, we have, on average, a residual of magnitude 1.5871 but with only 1 variablem, on average, we have a residual of 8.4947. Theoretically, with infinite (independent) variables, we might be able to achieve 100% accuracy on the training data; we will have overfit to the the training data's noise. However, generalizability will be low for the model.

Note: b0 represents the constant of the linear model, b1 represents the coefficient of x1, b2 the coefficient of x3, and so on.

p3

Exported 2/11/2026, 9:46:09 PM

In [2]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
from scipy.stats import t
import scipy.stats as stats
from scipy import io
import sympy as sp
import sys
import os
sys.path.append('../hw1')  # Add the directory containing heat_rod.py
from heat_rod import SteadyStateRod
```

In [3]:

```
data_7p7 = io.loadmat('exercise_7p7_data.mat')

data = data_7p7['copper_data']
x = data[:, 0]        # position (cm)
y_obs = data[:, 1]    # temperature (°C)

L = x.max()
k = 4.01  # W/cm/°C
data_7p7
x_data = data[:, 0]        # position (cm)
T_data = data[:, 1]        # temperature (°C)
```

In [4]:

```
# Model parameters for copper
a = 0.95  # cm
b = 0.95  # cm
L = 70.0  # cm
Tamb = 21.29  # °C
k = 4.01  # W/cm°C for copper (different from aluminum's 2.37)

# Initial parameter guesses
Phi_guess = -10.
h_guess = 0.001

# Create rod object with initial guesses
rod = SteadyStateRod(a=a, b=b, h_val=h_guess, k_val=k,
                     Phi_val=Phi_guess, Tamb_val=Tamb, L=L)

# Define the model function using the rod class
def model_func(params, x):
    """Model for steady-state temperature distribution"""
    Phi, h = params
    return rod.Ts(x, h=h, Phi=Phi)

def residuals(params, x, y):
    return model_func(params, x) - y

# Perform least squares optimization
result = least_squares(residuals, [Phi_guess, h_guess], args=(x_data,
T_data))
Phi_star, h_star = result.x

print("=" * 70)
print("COPPER ROD PARAMETER ESTIMATION")
print("=" * 70)
print(f"Optimal parameters:")
print(f"Phi* = {Phi_star:.4f} W/cm²")
print(f"h* = {h_star:.6f} W/cm²°C")

T_pred = model_func([Phi_star, h_star], x_data)

residuals_val = T_data - T_pred

n = len(x_data)  # 15
p = 2  # number of parameters
SSE = np.sum(residuals_val**2)
MSE = SSE / (n - p)  # Mean squared error
sigma = np.sqrt(MSE)  # Residual standard error

print(f"\nError statistics:")
```

```
print(f"Residual sum of squares (SSE): {SSE:.4f}")
print(f"Mean squared error (MSE) = {MSE:.4f}")
print(f"Residual standard error σ = {sigma:.4f} °C")
```

```
======================================================================
COPPER ROD PARAMETER ESTIMATION
======================================================================
Optimal parameters:
Phi* = -9.6110 W/cm²
h* = 0.001329 W/cm²°C

Error statistics:
Residual sum of squares (SSE): 0.3581
Mean squared error (MSE) = 0.0275
Residual standard error σ = 0.1660 °C
```

In [5]:

```
#  Calculate sensitivity matrix X (Jacobian) using the rod class
# X has dimensions n × p, where p=2 (Phi, h)
X = np.zeros((n, 2))

# Get sensitivity derivatives at optimal parameters
dT_dPhi = rod.dTs_dPhi(x_data, h=h_star, Phi=Phi_star)
dT_dh = rod.dTs_dh(x_data, h=h_star, Phi=Phi_star)

X[:, 0] = dT_dPhi  # Column for Phi
X[:, 1] = dT_dh    # Column for h

# Compute covariance matrix
XtX = X.T @ X
XtX_inv = np.linalg.inv(XtX)
var_b = MSE * XtX_inv  # Covariance matrix
std_b = np.sqrt(np.diag(var_b))  # Standard errors

print(f"\nCovariance matrix:")
print("V = ")
print(var_b)

print(f"\nStandard errors:")
print(f"σ_Φ = {std_b[0]:.4f}")
print(f"σ_h = {std_b[1]:.8f}")

z_critical = stats.norm.ppf(0.975)  # 1.96
CI_95 = np.column_stack([
    np.array([Phi_star, h_star]) - z_critical * std_b,
    np.array([Phi_star, h_star]) + z_critical * std_b
])

print("\n" + "=" * 70)
print("95% CONFIDENCE INTERVALS (z = 1.96, normal approximation)")
print("=" * 70)
print(f"{'Parameter':<10} {'Lower Bound':<15} {'Upper Bound':<15}")
print("-" * 70)
print(f"{'Φ':<10} {CI_95[0, 0]:<15.4f} {CI_95[0, 1]:<15.4f}")
print(f"{'h':<10} {CI_95[1, 0]:<15.8f} {CI_95[1, 1]:<15.8f}")

# Also calculate 2-sigma intervals (approximately 95% for large n)
print("\n" + "=" * 70)
print("2-SIGMA CONFIDENCE INTERVALS (~95.4% for normal)")
print("=" * 70)
CI_2sigma = np.column_stack([
    np.array([Phi_star, h_star]) - 2 * std_b,
    np.array([Phi_star, h_star]) + 2 * std_b
])
```

```
print(f"{'Parameter':<10} {'Lower Bound':<15} {'Upper Bound':<15}")
print("-" * 70)
print(f"{'Φ':<10} {CI_2sigma[0, 0]:<15.4f} {CI_2sigma[0, 1]:<15.4f}")
print(f"{'h':<10} {CI_2sigma[1, 0]:<15.8f} {CI_2sigma[1, 1]:<15.8f}")
```

```
Covariance matrix:
V =
[[ 4.12115156e-03 -5.35583166e-07]
 [-5.35583166e-07  7.52299545e-11]]

Standard errors:
σ_Φ = 0.0642
σ_h = 0.00000867

======================================================================
95% CONFIDENCE INTERVALS (z = 1.96, normal approximation)
======================================================================
Parameter  Lower Bound     Upper Bound
----------------------------------------------------------------------
Φ          -9.7369         -9.4852
h          0.00131225      0.00134625

======================================================================
2-SIGMA CONFIDENCE INTERVALS (~95.4% for normal)
======================================================================
Parameter  Lower Bound     Upper Bound
----------------------------------------------------------------------
Φ          -9.7394         -9.4826
h          0.00131190      0.00134660
```

3.2

In [6]:

```
residuals = T_data - T_pred  # υ - y(q)
RMS_residuals = np.sqrt(np.mean(residuals**2))

print("\n" + "=" * 70)
print("RESIDUAL ANALYSIS")
print("=" * 70)
print(f"Root-Mean-Square (RMS) of residuals: {RMS_residuals:.6f} °C")
```

```
======================================================================
RESIDUAL ANALYSIS
======================================================================
Root-Mean-Square (RMS) of residuals: 0.154520 °C
```

3.3

In [13]:

```
sigma_obs = np.sqrt(MSE)

# Estimated standard deviations from sampling distribution (parameter
std errors)
sigma_Phi = std_b[0]
sigma_h = std_b[1]

print("\n" + "=" * 70)
print("3.3) ESTIMATED STANDARD DEVIATIONS")
print("=" * 70)
print(f"\nA) Standard Deviation of Observation Errors:")
print(f"   σ_obs = = √({MSE:.6f})")
print(f"         = {sigma_obs:.6f} °C")
print(f"   Typical magnitude of measurement/observation error")

print(f"\nB) Standard Deviations from Sampling Distribution (Parameter
Uncertainty):")
print(f"   σ_Φ (for Φ) = {sigma_Phi:.6f} W/cm²")
print(f"   σ_h (for h) = {sigma_h:.10f} W/cm²°C")
print(f"   Uncertainty in estimated parameters due to limited data")
```

```
======================================================================
3.3) ESTIMATED STANDARD DEVIATIONS
======================================================================

A) Standard Deviation of Observation Errors:
   σ_obs = = √(0.027550)
         = 0.165981 °C
   Typical magnitude of measurement/observation error

B) Standard Deviations from Sampling Distribution (Parameter
Uncertainty):
   σ_Φ (for Φ) = 0.064196 W/cm²
   σ_h (for h) = 0.0000086735 W/cm²°C
   Uncertainty in estimated parameters due to limited data
```

3.4

In [8]:

```
# Plot results
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Plot 1: Model fit
ax1 = axes[0]
x_fine = np.linspace(10, 66, 200)
T_fine = model_func([Phi_star, h_star], x_fine)

ax1.plot(x_fine, T_fine, 'b-', label='Model', linewidth=2)
ax1.plot(x_data, T_data, 'rx', markersize=6, markeredgewidth=1.5,
label='Data')
ax1.set_xlabel('Distance (cm)', fontsize=12)
ax1.set_ylabel('Temperature (°C)', fontsize=12)
ax1.set_title(f'Copper Rod: Model Fit\nΦ*={Phi_star:.2f}, h*=
{h_star:.6f}', fontsize=12)
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3)

# Plot 2: Residuals vs x
ax2 = axes[1]
ax2.plot(x_data, residuals_val, 'bo', markersize=6,
markeredgewidth=1.5)
ax2.axhline(y=0, color='r', linestyle='--', linewidth=1.5)
ax2.set_xlabel('Distance x (cm)', fontsize=12)
ax2.set_ylabel('Residuals (°C)', fontsize=12)
ax2.set_title('Residuals vs Position', fontsize=12)
ax2.grid(True, alpha=0.3)

# Add ±2σ bands to residuals plot
ax2.axhline(y=2*sigma, color='gray', linestyle=':', linewidth=1,
alpha=0.7)
ax2.axhline(y=-2*sigma, color='gray', linestyle=':', linewidth=1,
alpha=0.7)
ax2.text(x_data[-1], 2*sigma, '  +2σ', fontsize=9,
verticalalignment='bottom')
ax2.text(x_data[-1], -2*sigma, '  -2σ', fontsize=9,
verticalalignment='top')

plt.tight_layout()
plt.show()
```